

Program 1

2nd & Smallest array

#include <stdio.h>

int arr[10];

void main() {

int a, s1, s2;

printf("Enter array element: \n");

for (int i = 1; i < n; i++) {
scanf("%d", &a);
arr[i] = a;

}

s1 = arr[0];

for (int k; k < 7; k++) {
if (arr[k] == s1) {
continue;} else if (arr[k] < s2) {
s2 = arr[k];

}

}

Print 1st & 2nd smallest ele: 11.
printf("%d", n2);

}

o/p

Enter array

1 2 3 4 5 6 7

Re 2nd smallest element in the array is 2

8) Find the total number of duplicate elements

if include < 100000
int main() {

int arr[100];

int size, i, j, count = 0;

printf("Enter size of the array: ");

scanf("%d", &size);

printf("Enter array elements: ");

printf("Enter elements in array: ");
for(int i=0; i<size; i++) {

scanf("%d", &arr[i]);

}

for (int i=0; i<size; i++) {

for (j=i+1; j<size; j++) {

if (arr[i] == arr[j]) {

count++;

break;

}

}

printf("Total number of duplicate elements = %d", count);

return 0;

}

time = $5 \times [\text{shoetst}]$;
 $\text{cl}[\text{shoetst}] = \text{time}$;
 $\text{used}[\text{shoetst}] = 1$;
 completed++;

$\text{tot}[\text{shoetst}] = \text{cl}[\text{shoetst}] \times \text{shoetst}$;
 $\text{cut}[\text{shoetst}] = \text{tot}[\text{shoetst}] - 10 \times \text{shoetst}$;
 $\text{wt}[\text{shoetst}] = \text{wt}[\text{shoetst}]$;

$\text{sumwt} += \text{tot}[\text{shoetst}]$;
 $\text{sumwt} += \text{cut}[\text{shoetst}]$;
 $\text{sumwt} += 1 \times [\text{shoetst}]$;

$\text{avg wt} = (\text{load}) \times \text{sumwt} / n$;
 $\text{avg wt} = (\text{load}) \times \text{sumwt} / n$;

$\text{printf} (" \backslash n \text{ Total } + \text{wt} + \text{cut} + \text{TA} \backslash n "$
 $\text{t RTT})$;

$\text{for} (i=0, i < n, i++) \}$

$\text{printf} (" \backslash n \text{ Total } + \text{wt} + \text{cut} + \text{TA} \backslash n "$
 $\text{t RTT})$;

$\text{printf} (" \backslash n \text{ Total } + \text{wt} + \text{cut} + \text{TA} \backslash n "$
 $\text{t RTT})$;

$\text{printf} (" \backslash n \text{ Avg TA : } \% .2 \text{f} \backslash n "$
 $\text{avg wt} : \% .2 \text{f} \backslash n "$
 $\text{avg wt} : \% .2 \text{f} \backslash n "$
 $\text{printf} (" \backslash n \text{ Influence RT : } \% .2 \text{f} \backslash n "$
 $\text{avg wt} : \% .2 \text{f} \backslash n "$;

return 0;

$\text{avg wt} = (\text{load}) \times \text{sumwt} / n$;
 $\text{avg wt} = (\text{load}) \times \text{sumwt} / n$;

$\text{printf} (" \backslash n \text{ Total } + \text{wt} + \text{cut} + \text{TA} \backslash n "$
 $\text{t RTT})$;

$\text{for} (i=0, i < n, i++) \}$
 $\text{printf} (" \backslash n \text{ Total } + \text{wt} + \text{cut} + \text{TA} \backslash n "$
 $\text{t RTT})$;

$\text{printf} (" \backslash n \text{ Total } + \text{wt} + \text{cut} + \text{TA} \backslash n "$
 $\text{t RTT})$;

}

point ("N Average MT: 2N Average
wt: 2N Average CK: 1.2N")
avg, avg, avg, avg, avg,

average,

3

o/p

Enter number of process: 4

Enter AT & BT for P: 0 2

Enter AT and BT for P1: 1 2

Enter AT and BT for P2: 5 3

Enter AT and BT for P4: 6 7

P	AT	BT	CT	MT	WT	RT
P1	0	2	2	2	0	0
P2	1	2	4	3	1	1
P3	5	3	8	3	0	0
P4	6	4	12	6	2	2

Avg MT: 3.50

Avg WT: 0.15

9 sum of rows & columns

#include <stdio.h>
void main() {

int rows, cols, i, j, sum;
printf ("Enter rows, cols: ");
scanf ("%d %d", &rows, &cols);

int matrix[rows][cols];
printf ("Enter elements of the matrix: ");

for (i = 0; i < rows; i++) {

for (j = 0; j < cols; j++) {

for (k = 0; k < cols; k++) {
scanf ("%d", &matrix[i][j]);

}

printf ("\n Row sum: \n");

for (i = 0; i < rows; i++) {
sum = 0

for (j = 0; j < cols; j++) {

sum = 0

for (j = 0; j < cols; j++) {

sum += matrix[i][j];

}
printf ("sum of row %d = %d\n", i+1, sum);

}
printf ("\n Column sum: \n");
for (j = 0; j < cols; j++) {

sum = 0;

for (i = 0; i < rows; i++) {
sum += matrix[i][j];

}

printf("sum of %d to %d",
1, 1000000);

}
return 0;
}

2 Delete Array

#include <stdio.h>
void main()

{
int arr[100];

int size, pos, i;
printf("Enter size: ");

scanf("%d", &size);

printf("Enter elements: ");

for (int i = 0; i < size; i++)

scanf("%d", &arr[i]);
}

printf("Enter two pos to be deleted: ");
scanf("%d", &pos);

if (pos > size) {

printf("Invalid pos");
}

{ else {

for (i = 0; i < pos - 1; i++) {

while (i < size) {

arr[i] = arr[i + 1];

i++;

}


```

    print("array: ", n)
    for ("array: ", n)
    do {int i = 0; i < size - 1; i++}
    print("\n", array[i]);
  }

```

if can't find the second largest array.

```

    # include <stdio.h>
    # include <stdlib.h>

```

```

    int arr [17];
    void main()
    {

```

```

        int a, b, c;
        printf("Enter array elements: ");
        for (int j = 0; j < 7; j++)

```

```

        if (arr[j] > c)
        {
            c = arr[j];
        }
    }

```

```

    c2 = arr[0];
    for (int k, k < 1; k++)
    {
        if (arr[k] > c2)
        {
            continue;
        }
    }

```

```

    }
    printf("our 2nd largest is: ");
    c2 = arr[k];

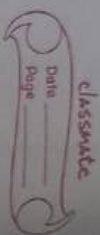
```

```

    }
    printf("our 2nd largest is: ");
    printf("%d", c2);

```

WEEK-1



1) FCFS

#include <stdio.h>

int main() {

int n, i, sum1, sum2;
printf("Enter number of processes: ");

scanf("%d", &n);

int a[n], b[n], c[n], d[n], e[n],
o[n];

Read array a, array b, array c,

sum1 = sum2 = sum3 = 0;

for (i = 0; i < n; i++) {

printf("Enter Process time & round");

printf("Process time: ");

scanf("%d", &a[i]);

}

a[0] = a[0] + b[0];

for (i = 1; i < n; i++) {

if (i == 1, i < n, i++) {

c[i] = a[i] + b[i];

else {

c[i] = a[i] + b[i];

for (i = 0; i < n; i++) {

printf("Process time: ");

scanf("%d", &a[i]);

printf("Process time: ");

scanf("%d", &a[i]);

printf("Process time: ");

2)

534

non-presumptive

Hin duale $\langle \text{Hed}^{\circ} \cdot h \rangle$

last move () {

int n, f, m = 0; // completed = 0, shares

parent (11/11/1987)
 Secord (11/11/1987)

$\vec{a}(n), \vec{b}(n), \vec{c}(n), \vec{f}(n), \vec{u}(n)$

$$g \in \text{Int}(\mathbb{A})$$

but verified (n)

not written $= 0$, sum of ≥ 0 , sum of ≥ 0 , sum of ≥ 0

Heart angiot, angiot angiot,

$$f_{\text{pol}}(i=0; \text{KR}, i++) \&$$

Print Pl Center Arrival time &
Buss time p/d 11/14/15

Built June 1st 1866 E.H.

Scung (1.2 / 2 / 2.2 / 3)

$$\text{unified } \Pi_1 = 0$$
$$\text{while (completed} < n) \{$$

Most left = 1

$$\text{for } (1 \geq 0, 1 \leq n, 1 \leq r) \}$$

1st 1st June 1881

$$y(a) = y(b) = 0$$

d) $(\text{Shadyside} = -1) \wedge (\text{Littleside} = 1)$

$\text{Sensitivity} = \frac{TP}{TP + FN}$

9

1) $(\text{phase} = -1)$

first,

Coastline;

9

3) Permutation STP

#include <stdio.h>
int main()

int n, i, time = 0, smallest, count = 0;
printf("Enter number of process:");

scanf("%d", &n);

int a[n], bt[n], ct[n], pt[n],
wt[n], start[n];

float avgat, avgwt, avgct;

for (i = 0; i < n; i++)

printf("Enter AT & BT for
P%d: ", i+1);

scanf("%d %d", &at[i], &bt[i]);

if (i % 2 == 0) {

start[i] = -1;

classmate
Date
Page

while (count < n) {

smallest = -1;

for (i = 0; i < n; i++) {

if (at[i] < time & bt[i] > 0) {

if (smallest == -1 || at[i] < at[smallest])

smallest = i;

}

if (smallest == -1) {

time = -1;

continue;

} (start[smallest] == -1)
start[smallest] = time;

time = time + bt[smallest];

classmate
Date
Page

Week-2
Priority

Non-Pre-emptive

#include <stdio.h>

int main() {

int n;

printf("Enter number of processes: ");
scanf("%d", &n);

int arr[10], b[10], p[10],
int waiting, total, rt[10], ct[10], done[10];

for (int i = 0; i < n; i++) {

printf("Enter AT of P%d: ", i);
scanf("%d", &arr[i]);

printf("Enter BT of P%d: ", i);
scanf("%d", &b[i]);

printf("Enter Priority of P%d: ", i);
scanf("%d", &p[i]);

done[i] = 0;

int time = 0, completed = 0;
while (completed < n) {

int best = 9999, index = -1;

for (int i = 0; i < n; i++) {

if (arr[i] <= time & b[i] <= 0 & done[i] == 0 & p[i] < best) {

best = p[i];
index = i;

}

if (index == -1) {

time++;
continue;

}

3

for $\{int\ i = 0; i \leq n; i++\}$

 $\{ \text{fat}(C1), \text{fat}(C2) \}$

5

6/23

Enter BT of P1: 5

Enter priority of P1: 2
Enter priority of P2: 3

Enter Priority of P2: 3

Enders AT 09:34
Enders AT 09:34

Enter 1st of Apr: 6
Enter priority: 2

PATRIOT

P1 1 5 2 0 5 0

P2	3	4	3	2	1	3	2
----	---	---	---	---	---	---	---

P3 4 6 2 2 8 2

Aug WT : 3.67

Aug TAT 5.67

Ang RT : 367

Pre-emptive

int main() {

int h;

printf("Enter number of processes :");
scanf("%d", &h);

int a[n], b[n], p[n];

int r[n], w[n], f[n], id[n], start[n];

for(int i=0; i<n; i++) {

printf("Enter AT of p%d", i+1);
scanf("%d", &a[i]);

printf("Enter BT of p%d", i+1);
scanf("%d", &b[i]);

printf("Enter priority of p%d", i+1);
scanf("%d", &r[i]);

g++ c1.c -o b1.c1
./b1.c1

int time = 0, completed = 0;
while (completed < n) {

int best = 999, idx = -1;
for(int i=0; i<n; i++) {

if (a[i] < time & b[i] > 0 & r[i] < best) {

best = a[i];
idx = i;

if (idx == -1)

{
time++;
continue;

if (start[idx] == -1) {
start[idx] = time;

r+ [idx] = -1;
time++;

$$d(x \mid dy) = 0$$

completed + +;

$\text{Affix} = \text{time}$

$$\text{fat}(\text{idx}) = \text{fat}(\text{idx}) - \text{adj}$$

all 1,000 - 2,000 - 6,000

float output = 0, avg test = 0, avg opt = 0;

[illegible]

for (int i = 0; i < n; i++) {

int response = start [i]-all[i],

$$\text{avg wt} = w + (i,j)$$
$$\text{avg lat} + = \text{lat} + \text{sid}$$

argt + = response

$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & i \\ 0 & 1 \end{pmatrix}$

$$11 = 1, a^2(12), b^2(13), c^2(14), d^2(15), e^2(16), f^2(17), g^2(18), h^2(19), i^2(20), j^2(21), k^2(22), l^2(23), m^2(24), n^2(25), o^2(26), p^2(27), q^2(28), r^2(29), s^2(30), t^2(31), u^2(32), v^2(33), w^2(34), x^2(35), y^2(36), z^2(37),$$

7

painty (" in Aug wt = % 28", avg (10),
 painty (" Aug SAT = % 29", avg (10),
 painty (" Aug RT = % 29", avg (10),
 Return.

2

d/p

Enter number of protons: 3

Enter 17 of 21: 6

Enter Priority of P1: 3

Ends BT of P2:5

Enkele Polarität 8/Pr: 2
- 101 AT 8/Pr: 9

enter BT of 13.4

Enders Prioritizing app: 1

P AT BT PR WT TPT RT

P1 1 6 3 9 15 0

P2 2 5 2 0 5 0

P3 9 4 1 0 4 0

Avg wt: 3.000

Avg TAT: 8.00

Avg RT: 0.00

Round Robin

include <stdio.h>

struct process {

int pid, priority, ct, tat, wqt, q;

int main() {

int n, tq;

struct process p[20];

int queue[100];

int visited[20] = {0};

int head = 0, rear = 0;

int time = 0, completed = 0;

int order[100];

int timeMark[10];

int k = 0;

float avgat = 0, avgrt = 0;

printf("Enter the no. of process: ");

scanf("%d", &n);

```
for (int i = 0; i < n; i++) {
```

```
    p[i].pid = i+1;
```

```
    printf("\n process %d\n", i+1);
```

```
    printf("Arrival time : ");
```

```
    scanf("%d", &p[i].at);
```

```
    printf("Burst time : ");
```

```
    scanf("%d", &p[i].bt);
```

```
    p[i].rt = p[i].bt;
```

```
}

for (int i = 0; i < n-1; i++) {
```

```
    for (int j = i+1; j < n; j++) {
```

```
        if (p[i].at > p[j].at) {
```

```
            struct process temp = p[i];
```

```
            p[i] = p[j];
```

```
            p[j] = temp;
```

```
        }
```

```
        printf("\n Enter time q : ");
```

```
        scanf("%d", &tz);
```

```
        time mark[0] = 0;
```

```
        while (computed < n) {
```

```
            for (int i = 0; i < n; i++) {
```

```
                if (p[i].at <= time && visited[i] == 0) {
```

```
                    queue [rear++] = i;
```

```
                    visited[i] = 1;
```

```
                }
```

```
                if (front == rear) {
```

```
                    time++;
```

```
                    continue;
```

```
                }
```

```
                int idx = queue [front++];
```

```
                order [K] = p[idx].pid;
```



```

if (p[idx].rt > tq) {
    p[idx].rt -= tq;
    time += tq;
}

```

```

else {

```

```

    time += p[idx].rt;
    p[idx].rt = 0;
    p[idx].ct = time;
    p[idx].fat = p[idx].ct - p[idx].wt;
    avgwt += p[idx].wt;
    avgfat += p[idx].fat;
    computed++;
}

```

```

k++;
time mark[k] = time;
for (int i = 0; i < n; i++) {
    if (p[i].at < time && visited[i] == 0)
        visited[i] = 1;
}

```

```

if (p[idx].rt < 0)
    return 0;
}

```

```

printf("\n ... Round ...")
printf("\n Round ...")
for (int i = 0; i < k; i++)
    printf("%d", order[i]);

```

```

printf("\n");
for (int i = 0; i < k; i++)
    printf("%d", order[i]);

```

```

printf("\n\n");

```

```

printf("Process | AT | ET | CT | RT |")
for (int i = 0; i < n; i++) {
    printf("%d | %d | %d | %d | %d |",

```

```

        p[i].pid, p[i].at, p[i].et, p[i].ct, p[i].rt);
}

```

print "\n\nAverage TAT = %0.2f", averageTAT
print ("\n\nAverage WT = %0.2f", averageWT)
return 0

3

%p

100000

Enter number of processes: 3

Process 1

AT: 1

BT: 5

Process 2:

AT: 2

BT: 6

Process 3:

AT: 3

BT: 6

Enter Tg: 3

Round Robin Scheduling

Print Chart:

P1 | P2 | P3 | P1 | P2 | P3 |

0 4 7 10 12 15 18

Process AT BT CT TAT WT

P1 1 5 12 16 6

P2 2 6 15 17 7

P3 3 6 18 14 8

Avg TAT = 13.67

Avg WT = 7.00

For different Tq values

Tq = 3 sec

Avg TAT = 12.67

Avg WIT = 7.00

Tq = 5 sec

Avg TAT = 8.67

Avg WIT = 3.67

Tq = 7 sec

Avg TAT = 9.00

Avg WIT = 3.67

∴ For Tq = 5 the system is most efficient.

multitimed queue

#include <stdio.h>

struct process {

int id;
int at;

int bt;
int ct;

int wt;
int tat;

int type;
int dtime;

int main() {

int n;

printf("Enter number of process : ");
scanf("%d", &n);

struct process p[n];

for(int i=0; i<n; i++) {

p[i].id = i;
p[i].at = 0;

p[i].bt = 0;

```

printf ("Process %d\n", i);
printf ("Enter arrival time: ");
scanf ("%d", &ps[i].at);

```

```

printf ("Enter burst time: ");
scanf ("%d", &ps[i].bt);

```

```

printf ("Enter type (0=system, 1=user)");
scanf ("%d", &ps[i].type);

```

```

}
int completed = 0, time = 0;

```

```

while (completed < n) {

```

```

    int idx = -1;

```

```

    for (int priority = 0; priority <= 1; priority++)

```

```

    {
        for (int i = 0; i < n; i++)

```

```

            if (ps[i].done == 0)

```

```

                if (idx == -1)

```

```

                    {

```

```

                        if (idx != -1) break;

```

```

                        if (idx != -1) break;

```

```

                        if (idx != -1) {

```

```

                            time += ps[idx].bt;

```

```

                            ps[idx].at = time;

```

```

                            ps[idx].ct = ps[idx].cl - ps[idx].bt;

```

```

                            ps[idx].done = 1;

```

```

                            completed++;

```

```

                        } else {

```

```

                            time++;

```


classmate
Date _____
Page _____

$$\text{for } (\text{int } i = 0; i < n; i++)$$

re

part 1: $\frac{1}{2} \times 100 = 50\%$

15/11/14

23.5-1.2

pu. type = 0 / 1

pro. at,
Oct 14

14.7
15.7

15.13. wt

PLD hat 1

particular)

total TAT = 01.7

2. full. far,

print (width wt of 25", fold wt/n)

Aug 15
TAT time: 9:21 a.m. total

Jan. 0.

o/p

Enter number of processes :)

process

Enter ATI: 0

Enter BT: 6

curry $\text{Cyr} \rightarrow \text{Cyr} \therefore C$

Process

Enter Ar :

Empire BT: 6

Enter 1920:

Process 2

Surfer AT: 0

Enter BT: 5

Enter type: 1

ID Type AT RT LT HT TA

~~System 0 1 1 0 6~~

System	β	α	γ	δ	ϵ	ζ	η	θ	ι	κ	λ	μ	ν	ξ	$\omicron$	π	ρ	σ	τ	υ	ϕ	χ	ψ	ω
1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23

11 3 5

WT: 1.67

1. 8: 141

Rata Monotic

AND

Earliest Deadline First

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

```
#define MAX 10
```

```
typedef struct {
```

```
int id;
```

```
int burst;
```

```
int deadline;
```

```
int period;
```

```
int completion;
```

```
int waiting;
```

```
int turnaround;
```

```
} Process;
```

```
void sort EDF (Process P[], int n)
```

```
{
    Process temp;
    for (int i = 0; i < n - 1; i++)
```

```
{
    for (int j = i + 1; j < n; j++)
```

```
{
    if (P[i].deadline >
        P[j].deadline)
```

```
{
    temp = P[i];
    P[i] = P[j];
    P[j] = temp;
```

```
}
```

```
void sort RMS (Process P[], int n)
```

```
{
    Process temp;
```

```
for (int i = 0; i < n - 1; i++)
```


for (int j = i + 1; j < n; j++)
if (arr[i].priority > arr[j])

$$P_{\text{temp}} = P_{\text{res}}$$

great power (process) with

$$\lim_{x \rightarrow 0} f(x) = 0$$

for $(i, j) \sim (k, l)$

$$f_{BD} = f_{max}$$
$$\text{float val} = 0.0;$$

Path - (Post) PCI - bang

reborn with,

void calculateProcessPTS(int n)

rest time = 0;

for $\text{int } i = 0; i < n; i++) \{$

$$f_{\text{max}} = P(i) \cdot \text{burst}$$

211) - combustion - 4000,

complication

-Pill: burst

```

int main (c)
{
    process p[max], temp [max];
    int n;

    print ("Enter num. of process");
    scanf ("%d", &n);

    print ("Enter process id");
    for (int i=0; i<n; i++)
        print ("%d", i);

    print ("B1");
    scanf ("%d", &ps1);

    print ("Deadline (for EDT)");
    scanf ("%d", &ps1);

    print ("Total / for RM");
    scanf ("%d", &ps1);
}

```

```

for (int i=0; i<n; i++) {
    temp[i] = ps1;

    print ("In SDF");

    sort ps1 (ps1);

    dead-ctd = ps1[0];

    print ("CPU util %d", util);

    if (util <= 1) {
        print ("Schedule util <= 1");
        util;

        (calculate (n));

        print ("ED BT Deadline (over 7711)");

        for (int i=0; i<n; i++) {
            print ("%d", ps1[i]);
        }
    }
}

```



```

P[i].id,
P[i].burst,
P[i].deadline,
P[i].completion,
P[i].waiting,
P[i].turnaround,

printf("Rate mono tic"),
sort RMS(temp, n),
util = (pu util (temp, n))
printf("%d %d %d %d %d",
P[i].id,
P[i].burst,
P[i].deadline,
P[i].completion,
P[i].waiting,
P[i].turnaround,
return 0;

```

Op
No. of process : 3
P1
BF : 2
Deadline : 4
P2
BF : 3
Deadline : 7
P3
BF : 2
Deadline : 2

Rate monotonic

Process 1

BT: 2 Period: 5

Process 2:

BT: 2 Period: 10

Process 3:

BT: 3 Period: 20

EDF

CPU Util = 0.75

Schedulable: Util < 1

ID BT Deadline (15) w.r.t. 1

1	2	4	2	0	2
3	7	5	2	5	
2	8	7	5	7	

Rate monotonic

CPU Util = 0.75

RM Bound: 0.7500

Schedulable

ID BT Period (15) w.r.t. 1

1	2	5	2	0	2
2	2	10	4	2	4
3	3	20	7	4	7

Proportional Scheduling

if include < std::h>

if include < std::h>

if include < time.h>

struct Task {

int id, ticks;

};

int main() {

int n, total = 0, iterations;

printf ("Enter number of tasks: ");

scanf ("%d", &n);

struct task t[n];

for (int i = 0; i < n; i++) {

printf ("Enter ticks: ");

scanf ("%d", &t[i].ticks);

t[i].id = i + 1;

total += t[i].ticks;

}

printf ("Enter no. of Scheduling cycles: ");

scanf ("%d", &iterations);

sort(t, t + n);

for (int i = 0; i < iterations; i++) {

int winner = 0, ticks;

for (int j = 0; j < n; j++) {

sum += t[j].ticks;

if (winner == 0) {

printf ("Cycle %d: ", i);

select winner & break;

4
4
4
4
opp

enter no. of tests: 3

enter ticked for 1: 20

enter ticked for 2: 30

enter ticked for 3: 40

enter number of cycles: 5

cycle 0: test 2 selected

cycle 1: test 1 selected

cycle 2: test 3 selected

cycle 3: test 2 selected

cycle 4: test 3 selected

~~25/12/25~~

bounded Buffer

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
```

```
#define SIZE 5
```

```
int Buffer [SIZE];
```

```
int in = 0, out = 0;
```

```
int p_count, c_count;
```

```
sem_t mutex, empty, full;
```

```
void * producer (void * arg) {
```

```
    for (int i = 0; i < p_count; i++)
```

```
        sem_wait (&empty);
```

```
        sem_wait (&mutex);
```

```
    buffer [in] = i;
    printf ("Produced : %d at buffer\n", i, in);
```

```
    in = (in + 1) % SIZE;
```

```
    sem_post (&mutex);
    sem_post (&full);
```

```
    return NULL;
```

```
}
void consumer (void * arg) {
```

```
    for (int i = 0; i < (c_count + 1); i++)
```

```
        sem_wait (&full);
```

```
        sem_wait (&mutex);
```

```
        printf ("Consumed : %d\n", i, in);
```

int main() {

pthread_t, p, c;

printf ("Enter number of items
produced:");

scanf ("%d", & p-count);

printf ("Enter number of items
consumed:");

scanf ("%d", & c-count);

sem_init (&mutex, 0, 1);

sem_init (&empty, 0, 8, 32);

sem_init (&full, 0, 1);

pthread_create (&p, NULL, produce,

pthread_create (&c, NULL, consume,

NULL);

pthread_join (p, NULL);

pthread_join (c, NULL);

return 0;

}

013

Enter number of items to produce

Enter number of items to consume

Produced: 1 at buffer 00

Produced: 1 at buffer 01

Produced: 2 at buffer 02

Consumed: 0 from buffer 00

Consumed: 1 from buffer 01

Type

14-11-20

Dining Philosophers Problem

#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define MAX 10

int n;

sem_t forks [MAX];

void *philosopher (void *arg)

int id = * (int *) arg;

printf ("Philosopher %d is
thinking\n", id);

sleep(1);

printf ("Philosopher %d is
eating\n", id);

printf

sem of philosophers: 2

philosopher

fork 1
fork 0

fork 0 8 2

fork 0
fork 1

50

int main() {

pthread_t pt[MAX];
int idst MAX;

Print the stack number of philosophers
scanf("%d", &n);

for (int i=0; i<n; i++) {

sem_init(&forkst[i], 0, 1);

for (int i=0; i<n; i++) {

int (i) = 1;

pthread_create(&pt[i], NULL,

philosopher,
&idst[i]);

for (int i=0; i<n; i++)

pthread_join(pt[i], NULL);

O/P

Enter number of philosophers: 2

philosoper 0 is thinking
philosoper 1 is thinking

philosoper 0 picked up right fork 1
philosoper 0 picked up left fork 0

philosoper 0 is eating
philosoper 0 put down fork 0 & 1

philosoper 0 is thinking

philosoper 1 picked up right fork 0
philosoper 1 picked up left fork 1

philosoper 1 is eating

philosoper 1 is put down fork 1 & 0

philosoper 1 is thinking

Trace

Both thinking forks[0] = 1, fork[1]

0 picks left

fork[1] → 0

0 picks right

fork[0] → 0

seats

0 puts down forks

forks[0] → 1
fork[1] → 0

Bounded Buffer Trace

buffer = [_ _ _ _]

in = 0, out = 0

empty = 5 full = 0 mutex = 1

Step 1 producer

empty → 5

mutex = 0

buffer[0] = 0

in = 1

semp post (mutex) → 0 → 1

sem post (full) → 0 → 1

buffer = [0 _ _ _]

in = 1, out = 0

empty = 4, full = 1

① Producer

sem-wait (empty) $\rightarrow 4 \rightarrow 3$
sem-wait (mutex) $\rightarrow 1 \rightarrow 0$

Buffer [1] = 1

in = 2

sem-post (mutex) $\rightarrow 0 \rightarrow 1$

sem-post (full) $1 \rightarrow 2$

in = 2

if $in \geq 2$ empty $\rightarrow 3$

out $\rightarrow 0$

(empty) $\rightarrow 3 \rightarrow 2$
(mutex) $\rightarrow 1 \rightarrow 0$

in = 2

sem-post (mutex) $\rightarrow 1$
sem-post (full) $\rightarrow 2 \rightarrow 3$

② Consumer

Buffer = [0, 1, 2, ...]

in = 3, out = 0

empty = 2, full = 3

sem-wait (full) $\rightarrow 3 \rightarrow 2$
sem-wait (mutex) $\rightarrow 1 \rightarrow 0$

item = Buffer [0]
out = 1

sem-post (mutex) $\rightarrow 1$

sem-post (empty) $2 \rightarrow 3$

~~Consumer~~ [0, 1, 2, ...]

in = 3 Out = 1

full = 2 empty = 3

③ Consumer

sem-wait (full) $\rightarrow 2 \rightarrow 1$
sem-wait (mutex) $\rightarrow 1 \rightarrow 0$

if $in \geq 2$ Buffer [1]
out = 2

Semiprest (matrix) 1
Sem post (empty) 2 → 3

Booster - 1 - 2 - 3 - 4 - 5

Full → Empty → 4

Out → 2 in → 3

~~April 4~~

Banker's Algorithm

It includes < station >

int main () {

int n, m, k,

printf ("Enter number of processes")
scanf ("%d", &n);

printf ("Enter number of resources for")
scanf ("%d", &m);

int alloc [n][m], max [n][m], avail;
int need [n][m];
int j, k, i, s, seq [n];

printf ("Enter Allocation Matrix")
for (i = 0; i < n; i++) {
for (j = 0; j < m; j++) {
scanf ("%d", &alloc [i][j]);

}



for (i=0; i<n; i++) {
 cout << "Enter Number: ";
 cin >> a[i];
}

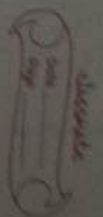
for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}



for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

for (i=0; i<n; i++) {
 cout << "Sum: " << a[i] << endl;
}

print ("n System is not state full")

break;

for (i=0; i<n; i++) {

print ("p%d", p[i]);

}

print ("n");

print ("n Maximal Matrix n");

for (i=0; i<n; i++) {

for (j=0; j<n; j++) {

print ("p%d", p[i][j]);

print ("n");

}

return;

q

Enter number of processes: 5

Enter number of resources: 10

Enter Allocation matrix

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter maximum matrix

7 1 3

3 2 2

4 0 2

2 2 2

4 3 3

Enter available resources:

4 3 2

System is self-study

Self Sequence: P1 P3 P4 P5 P2

Need Matrix

7 4 3

1 2 2

6 0 0

0 1 1

4 3 1

Dead lock

it includes <stdio.h>

int main () {

int n, m;

printf ("Enter number of processes: ");

scanf ("%d", &n);

printf ("Enter number of resources: ");

scanf ("%d", &m);

int allocation [n];

int request [n];

int available;

printf ("Enter allocation matrix: ");

for (int i = 0; i < n; i++)

for (int j = 0; j < m; j++)

scanf ("%d", &allocation[i][j]);


```

print("\nEnter Request Matrix: ")
for i in range(0, i < n):
    for j in range(0, j < m):
        s = input("Enter Available Resource: ")
    }
}

```

```

print("\nEnter Request matrix: ")
for i in range(0, i < n):
    for j in range(0, j < m):
        s = input("Enter Request Matrix: ")
    }
}

```

```

for i in range(0, i < n):
    for j in range(0, j < m):
        s = input("Enter Request Matrix: ")
    }
}

```

```

for i in range(0, i < n):
    for j in range(0, j < m):
        s = input("Enter Request Matrix: ")
    }
}

```

```

for i in range(0, i < n):
    for j in range(0, j < m):
        s = input("Enter Request Matrix: ")
    }
}

```

1 (good alternation)

length $S[j] = 1$
for

length $S[j] = 0$

}

not changed = 1

not changed {

changed = 0

for (int i = 0; i < m; i++) {

if (length $S[j] = 0$) {

not possible = 1;

for (int i = 0; i < m; i++) {

if (length $S[j] = 0$) {

possible = 0;
break;

}

}

if possible = 1 {

length "in blocks" of m

for (int i = 0; i < m; i++) {

cannot (if) + - alternating (i, j)

length $S[j] = 1$;

changed = 1;

}

}

}

end address = 0

length = 10 + 10 + 10 + 10 + 10 + 10 + 10 + 10 + 10 + 10

offset = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Address [0] = 0

Address [1] = 1

Address [2] = 2

?

Address [3] = 3

Address [4] = 4

?

?

0/0

Enter number of process

Enter number of resources

Enter Allocation matrix

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

0 0 0

0 0 0

1 0 0

0 0 0

0 0 0

Extra Available Memory

0 0 0

Process P0 can

Process P1 can

Process P2 can

Process P3 can

Process P4 can

Dead lock process, Above

10/10/20

Memory Allocation

Code

include <stdio.h>

define MAX 50

void firstFit (int blocks, int m, int

int allocation [MAX])

{ for (int i=0; i<n; i++)
allocation[i] = -1;

for (int i=0; i<n; i++) {

for (int j=0; j<m; j++) {

if (blocks[i] >= process[j]) {

allocation[i] = j;
blocks[i] -= process[j];
break;

}} }


```

print ("n --- First Fit ---")
printf ("Process No. | Process Size | Block No. |")

for (int i = 0; i < n; i++) {
    printf ("%d | %d | %d | %d |", i, process[i], allocation[i], -1);
}

if (allocation[i] == -1)
    printf ("%d | %d | %d | %d |", i, process[i], allocation[i], -1);
else
    printf ("Not Allocated\n");
}

void bestFit (int blocks[], int m,
int process[], int n) {
    int allocation[m][n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;

```

```

        for (int j = 0; j < m; j++) {
            if (blocks[j] >= process[i]) {
                if (bestIdx == -1 || blocks[j] < blocks[bestIdx])
                    bestIdx = j;
            }
        }

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blocks[bestIdx] = process[i];
        }
    }

    printf ("n --- Best Fit ---")
    printf ("Process No. | Process Size | Block No. |")
}

```



```

for (int i=0; i<n; i++) {
    printf("%d\n", i);
}
else printf("not allocated\n");
}
int main() {
    int m, n;
    int block[100], process[100];
    printf("Enter number of memory blocks,");
    scanf("%d", &m);
    printf("Enter size of memory blocks,");
    scanf("%d", &n);
}

```

```

for (int i=0; i<m; i++)
    scanf("%d", &block[i]);
printf("Enter number of processes,");
scanf("%d", &n);
printf("Enter sizes of processes,");
for (int i=0; i<n; i++)
    scanf("%d", &process[i]);
int b1[100], b2[100], b3[100];
for (int i=0; i<m; i++) {
    b1[i] = block[i];
    b2[i] = block[i];
    b3[i] = block[i];
}
}

```

first bit (b1, m, process, n).
 best fit (b2, m, process, n).
 worst fit (b3, m, process, n).

return;

O/P

Enter number of memory blocks: 5
 Enter sizes:

100 500 200 300 600

Enter number of processes: 4
 Enter sizes of processes

212 417 112 426

if (j == 0) {

pages [k] = pages [i];

k = (k+1) % 10;

faults++;

printf ("page %d -> ", pages[i]);

for (i = 0; i < j; i++) {

printf ("%d", pages[i]);

printf (" ", pages[i]);

} else {

~~printf ("page %d -> ", pages[i]);~~

}

#include <stdio.h>

void FIFO (int pages [], int n, int p)

{
int frames [p], i, j, k = 0; found = 0;

for (i = 0; i < n; i++)

frames[i] = -1;

printf ("n ----- FIFO ----- \n");

for (i = 0; i < n; i++) {

found = 0;

for (j = 0; j < p; j++)

{ if (frames[j] == pages[i])

found = 1;

break;

}

if (!found) {

frames[k] = pages[i];

k = (k + 1) % p;

faults++;

printf ("page %d -> ", pages[i]);

for (j = 0; j < p; j++)

printf ("%d ", frames[j]);

printf ("Fault \n");

} else {

~~printf~~ printf ("Page %d -> ", pages[i]);

}

printf ("old page fault (%p) : %d",
}

read (&ru / int pages, &utn, &rd);

ut = frames[0], &ru[0];

if (i, if found, faults = 0;

ut counters = 0, pos, min;

for (i = 0; i < ut; i++) {

if (frames[i] = -1;

frame[i] = 0;

}

printf ("n - - - - - &ru - - - - - n");

for (i = 0; i < n; i++) {

found = 0;

for (i = 0; i < ut; i++) {

if (frames[i] == pages[i]) {

counters++;
if (frames[i] == counters;
found = 1;
break;

}

}

if (found) {

min = frame[0];

pos = 0;

for (i = 1; i < ut; i++) {

if (frames[i] < min) {

min = frame[i];

pos = i;

}

counter = 1
 for each (obj : pages[i],
 for (pos : counter,
 faults++)

print ("Page %d = ", pages[i])
 for (i = 0; i < j; i++)

print ("Page %d = ", pages[i])
 print ("Faults")

for {

print ("Page %d = ", i)

}

print ("Total Page Faults")

}

faults



void OPTIMAL (int pages[], int n, int f)
 int front = 0;
 int i, k; found faults = 0;
 int pos; frontmost;

for (i = 0; i < j; i++)
 pages[i] = -1;

print (" ", n, " ... OPTIMAL ... ", n);

for (i = 0; i < n; i++) {

found = 0

for (j = 0; j < j; j++) {

if (pages[j] == pages[i])

found = 0;

break;

}

}

} (j, forward) {

pos = -1;

for (forward = -1;

for (i = 0; i < n; i++) {

int mid < i;

for (i = 0; i < n; i++) {

int mid < i;

for (k = i + 1; k < n; k++) {

} (j, forward) {

int mid < i;

break;

}

} (j, forward) {

pos = -1;

break;

} (j, forward) {

pos = -1;

} (j, forward) {

pos = -1;

break;

pos = -1;

pos = -1;

pos = -1;

pos = -1;

pos = -1;

pos = -1;

Print ("Error Page fault (optimal)",

%d\n", fault,

int main() {

int pages (10, n, l);

Print ("Enter number of pages:");

scanf ("%d", &n);

Print ("Enter page references (n):");

for (i=0; i<n; i++)

array (i, &pages[i]);

Print ("Enter number of frames:");

FIFO (pages, n);

LRU (pages, n);

Optimal (pages, n);

return 0;

}
OP

Enter number of pages -
Enter page references -

10 1 2 0 3 0 4 2 3 0 3

--- FIFO ---

Page 7 -> 7 - 1 - 1 - 1 (Fault)

Page 0 -> 7 10 - 1 (Fault)

Page 1 -> 7 0 1 (Fault)

Page 2 -> 10 1 (Fault)

Page 0 -> 1 fault

Page 3 -> 2 3 1 (Fault)

Page 0 -> 2 3 0 (Fault)

Page 4 -> 4 3 0 (Fault)

Page 2 -> 4 0 (Fault)

Page 3 -> 4 0 3 (Fault)

Page 0 → 0 1 1 (Fault)

Page 30 not

Total Page Faults (PFO) = 10

--- PFO ---

→ 1-1-1 (Fault)

Page 0 → 10-1 (Fault)

Page 1 → 70-1 (Fault)

Page 2 → 20-1 (Fault)

Page 3 → 10-1 (Fault)

Page 4 → 20-1 (Fault)

Page 5 → 10-1 (Fault)

Page 6 → 90-1 (Fault)

Page 7 → 90-1 (Fault)

Page 3 → 43-1 (Fault)

Page 0 → 05-1 (Fault)

Page 3 → not

Total Page Faults (PFO) = 1

OPTIMAL:

Page 1 → 1-1-1 (Fault)

Page 0 → 0-1-1 (Fault)

Page 1 → 0-1-1 (Fault)

Page 0 → not

Page 3 → 02-1 (Fault)

Page 0 → not

Page 4 → 02-1 (Fault)

Page 2 → not

Page 3 → not

Page 0 + 0 + 3 (Pounds)
1943 - put

Total Page Book A: 7

~~Page 1943~~

INDEX

NAME: OmKorish

STD.: _____

SEC.: _____

ROLL NO.: 201327 SUB.: OS Obs

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.		Week - 0		
2.		Week 1 - 1 (Fict & SFT)		
3.		Week - 2 (priority Scheduler Preemptive Nonpreemptive)		<u>1/13/23</u>
4.		Round Robin		
5.		Multilevel queue		<u>2/24/23</u>
6.		Rate monotonic	10	
7.		Earliest Deadline First		<u>5/14/23</u>
8.		Proportional Scheduling		<u>8/4/26</u>
9.		Bounded Buffer		<u>2/24/24</u>
10.		Dining Philosopher		
11.		Banker Algorithm		<u>5/5/20</u>
12.		Deadlock Detection		

"Whenever you see 'cover', it means both the front and back."
"This puzzle can have multiple answers."

Answers
(1) 2, 3 (15, 3) 3, 4) 1, 5) 20, 6) 28

GKCM32510 P